Introduction

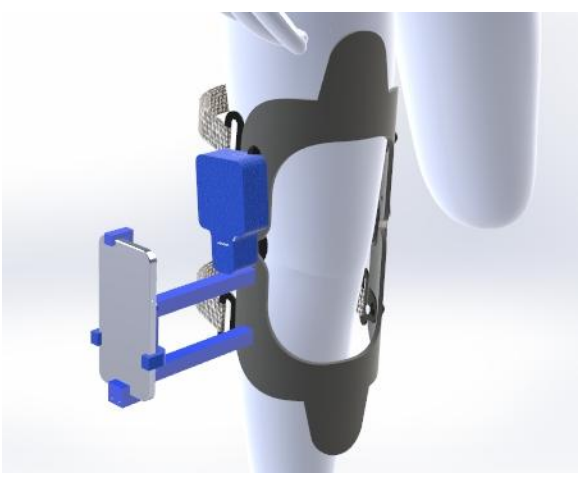
Patellofemoral pain is experienced by around 23% of the general population during their lifetime (1), with potential complications including cartilage damage and arthritis (2). A leading cause of this pain is patella maltracking (3), in which the patella deviates from normal translational or rotational motion (4). While orthopaedics frequently diagnose patella maltracking, quantification of its motion and a description of normal motion remain underdefined (5). There is significant interest in the development of a device to measure patella motion, to produce more accurate patient diagnoses.

For medical devices to be lawful and marketable, they must adhere to strict regulatory laws. In the UK, medical devices must comply with the Medical Device Regulations Act of 2002 to minimise potential risks (6). This legislation defines device classification, safety and traceability (6). Additionally, device compliance is supported by international standards, particularly ISO 14971, detailing systematic hazard identification and risk management across product lifecycle (7,8).

This report aims to present the iterative design process, prototype development and preliminary testing of a device to measure patellar motion *in vivo* when a patient is wearing a knee brace. The project investigates the use of a time-of-flight sensor to measure the mediolateral position of the patella relative to the transepicondylar axis (TEA) of the femur between 0-90º flexion. Through testing, the report seeks to validate a design aligned with international standards, to support clinical assessments of maltracking.

Proposed Design

The proposed design is shown in Figures 1, 2 and 3. The design was developed around an Ossur CTI3 Brace, with calf diameter of 164±5mm and thigh diameter of 176±5mm to appropriately fit a wide range of users. using the time-of-flight sensors of an iPhone 14 Pro. The concept was refined through an iterative process, following the specifications highlighted in the design specification, Table 1, and Failure Modes and Effects Analysis (FMEA), Table 2. Testing and validation of individual components was carried out to ensure that the design was suitable for required function.





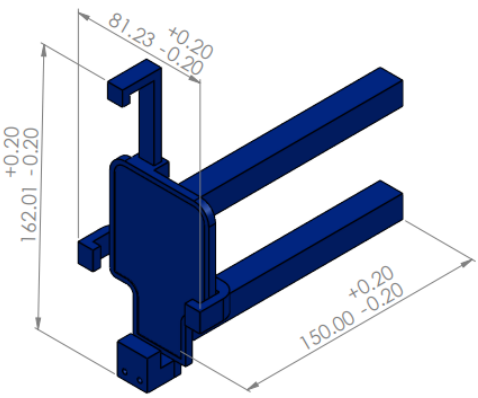


Figure 1 - CAD design of device; Figure 2 - Exploded CAD design of device; Figure 3 – Dimensions and tolerances of the phone mount (human body model by Alex Alex on GrabCAD; all other images produced by authors)

Table 1 – Design Specification for Proposed Device

User	Need	Implication	W	References	Verification	Design Requirement
Clinician	Quantify patella motion	Patella position measurable at a given time	5	Systematised MRI evaluation method (9)	Compare measured trajectory to MRI and motion capture data	Patella tracking accuracy ±3mm over full range of motion
Clinician	Full flexion motion capture	Knee within FOV and minimum depth	5	ISO 20685-1; Intel LiDAR Data Sheet (10, 11)	Minimum Separation = 15cm; Trigonometry at FOV	Sensor distance > 15cm
Clinician	Record flexion angle	Real-time joint angular measurement	4	Encoder Datasheet (12)	Calibrate potentiometer at 0º, 45º and 90º	Calibrated potentiometer and microcontroller
Clinician	Clear data display	Interpretable real-time output	4	ISO 9241-210 (13)	Practitioner feedback; Iterative design	Display of angle and patella position (<0.5s latency)
Clinician	Hygienic and reusable	Disinfectant resistant; Cleanable	4	ISO 17664-1; Datasheets (14,15)	Disinfectant exposure testing	Detachable, components can be disinfected
Clinician	Hinge aligned to TEA	Alignment identifiable and testable	4	Anatomical Literature (16)	Visual marking; Alignment jig test	Hinge alignment within ±2mm of TEA
Clinician	Measure across flexion	Hinge moves at least 90º	3	-	Angular limit testing	LED indicates rotation beyond 0-90º
Patient	Universal, comfortable fit	Accommodate 5th – 95th percentile	4	BS EN ISO 7250 (17)	Thigh and calf circumference measurement	Adjustable range of ±30mm for both circumferences
Patient	Prevent translation	Maintain fixation	4	-	Dynamic fitting test over full range of motion	Velcro fixation straps
Patient	Non-invasive	No irritation or skin damage	4	BS EN ISO 10993-10; Datasheets (15,18)	Abrasion tests; Soft tissue phantom; Swelling check	Hypoallergenic material
Patient	Structural integrity	Withstand peak stresses	3	BS EN ISO 10328 (19)	Four point bending test; Physical stress test	Yield strength > 2 x Expected Operating Stress
Patient	Free movement	Consider biomechanics	3	ISO 6385 (20)	Gait comparison with and without brace	Mass < 0.5kg
Hospital	Traceability	Product ID	5	ISO 13485 (21)	ID readability check	Unique ID number
Hospital	Minimal training	Clear usage guidance	2	IEC 62366 (22)	Practitioner feedback; Iterative design	Concise step-by-step guide
Hospital	Easy end-of-life	Separable components	2	ISO 15223-1 (23)	Timed dismantling	Modular, separable design

Table 2 – Failure Modes and Effect Analysis for Device; showing Severity, Probability and Risk Priority Numbers

Hazard	Effect	Causes	S	P	RPN	Reference	Recommended Mitigations	Validation	S	P	RPN
Software false output	Misdiagnosis	Inadequate or over filtering of data	3	4	12	ISO 62304 (24)	Apply frequency filtering; validate data against optical tracker	Ensure device accuracy ±3mm compared to optical tracker	3	2	6
Excessive strap pressure	Numbness, swelling, irritation	Overtighten brace straps by user	2	3	6	IEC 60601-1 (25)	IFU guidance on strap tightness	Comfortable Pressure: 10kPa	2	1	2
Bracket movement during motion	Invalid coordinate and position output	Device migration during knee flexion	5	3	15	ISO 14971 (26)	Reduce mass and centre of mass; minimise length of bracket	Repeatability testing across flexion cycles	5	2	10

Battery overheating	Potential burns or electrical fire	Short circuiting, PCB fault	5	3	15	IEC 62133 IEC 60601 (25, 27)	Thermal testing; replace battery with USB cable	Electrical safety testing under simulated conditions	4	1	4
Adverse skin reactions	Dermatitis, irritation	Material sensitivity,	3	3	9	ISO 10993(18)	Hypoallergenic; cleaning guidance	Cytotoxicity testing for prolonged exposure	3	1	3
Current leakage	Electrical shock risk	Insulation failure	4	3	12	IEC 60601-1; ISO 10993 (18, 25)	Redesign housing; use high resistivity material	Electrical safety testing under simulated conditions	4	1	4
Output misinterprets	Incorrect positioning data	Misaligned, calibration error	4	3	12	ISO 13485; ISO 14971 (21, 26)	Improved IFU, clinician training, useability testing	Formative useability validation passed	3	2	6
Mechanical failure	Patient injury, device breakage	Bracket fracture	3	2	6	ISO 14971 (26)	Fatigue testing, load-bearing specification	Mechanical endurance testing over 100 flexion cycles	3	1	3
Reuse contaminant	Infection transmission	Inadequate cleaning procedure	2	2	4	ISO 15883 (28)	Defined cleaning protocol	Cleaning efficacy testing	1	1	1

Material Specification & Sterilisation

The device will be manufactured from fused deposition modelled (FDM) polylactic acid (PLA). Table 3 details the key properties of FDM PLA, meeting the proposed requirements detailed in Table 1. In accordance with the risk mitigation procedures highlighted in Table 2, and ISO 10993, PLA must undergo cytotoxicity, sensitisation and irritation tests to ensure its applicability to the device (18).

Table 3 – Properties of FDM PLA at 100% infill.

Young's Modulus [MPa]	**Ultimate Tensile Strength [MPa]**	**Yield Stress [MPa]**	**Electrical Resistance [Ohms]**	**Cost [GBP/kg]**
3039 (29)	48 (29)	26 (29)	1.16 x 10^14 (30)	10 (31)

Under ISO 14937 and ISO 17664-1, a Class Im skin-contacting device does not require sterilisation but must be amenable to disinfection (14,32). ISO 15883-1 requires specification of validated disinfection protocols (28). Table 4 demonstrates testing of medical grade disinfectant solutions. While 0.85% sodium hypochlorite achieved complete microbial reduction, it caused surface degradation. Ethanol achieved 98% disinfection with no observed material damage; therefore, it is specified as the recommended disinfectant. Infill density will be set to 100% to minimise porosity and prevent disinfectant from becoming trapped within the material.

Table 4 – Efficacy and Long-Term Effects of Common Hospital Disinfectants (33)

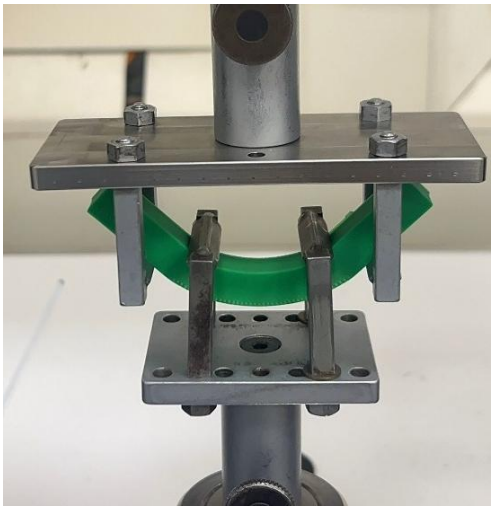
	Efficacy of Disinfectant (%)	**Effects of Prolonged Use (~24 hours)**
96% Ethanol	98	None Observed
70% Isopropanol	87	Melting of Structure
0.85% Sodium Hypochlorite	100	Surface and Filament Damage

Testing & Validation

To ensure the safety of a 3.7V lithium-ion battery, a test was undertaken to consider the surface temperature of the battery over time during charging. Using a multimeter, the battery was found to have a voltage of

3.9V. According to safety limitations, it could be charged up to 4.2V. After 6 minutes, the battery surface temperature was 45ºC, measured using a temperature probe. This would be unsafe during operation, as at the risk of overheating was too great, and the test was terminated for safety reasons. It was decided that a USB cable will be used to power the Arduino, despite the increased trip hazard, to minimise risk to the patient and comply with safety standards.

A four-point bending test was performed to determine the yield stress of a 10x10x100mm sample of FDM PLA, at 100% infill density. The test was carried out on an Instron 3000. The result after progressively applying load up to 2kN is shown in Figure 4, the flexural yield stress was determined to be ~75MPa, Figure 5. The operating stress of 0.136MPa (Appendix 2) is significantly below the yield stress with a safety factor of 10, confirming the safety of the device.



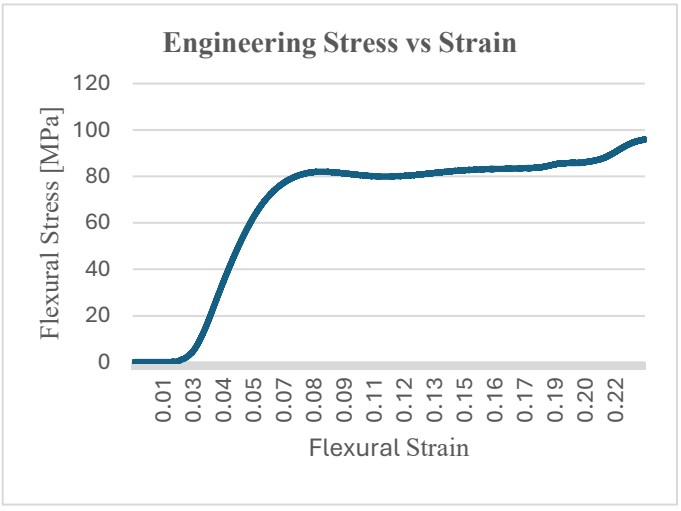


Figure 4 – Instron loaded with PLA sample after testing; Figure 5 - Graph showing engineering stress vs strain

Device Classification & Governing Regulations

The proposed design is a non-invasive, transient device intended for continuous use of less than 60 minutes. Under Annex IX of the Medical Devices Regulations Act of 2002, it is classified as a Class Im (measuring) device as it provides quantitative data to characterize patella movement, with intention to diagnose maltracking. In accordance with Annex I, a conformity assessment by a notified body is required to verify the accuracy of the system from sensor input to digital output. Annex II requires mitigation of risks that could compromise safety or performance. The device will also need to undergo UKCA certification to evidence that it meets requirements (34).

The manufacturer must demonstrate that the device performs its function while minimising risk. To ensure the device produces accurate diagnoses, the data collection and software must undergo verification and validation. To reduce error and improve the user experience, the design incorporates LED indicators to provide visual feedback. One LED indicates that the coloured markers have been identified, and the patient can begin rotating their leg. The second, linked to the rotary encoder, indicates the final angle at which the

patient should stop rotation. This prevents measurements being taken from an uncalibrated reference, leading to incorrect completion of the diagnostic function.

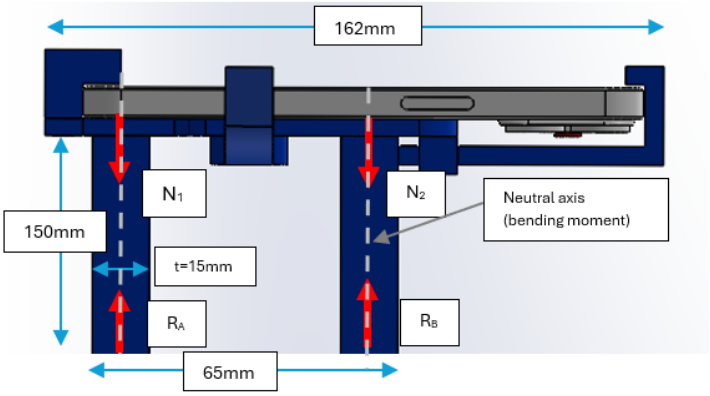


Figure 6 – Free body diagram showing forces acting on mounting device

Figure 6 shows a free body diagram of the forces acting on the mounting device. N1 and N2 are normal reaction forces, while Ra and Rb are supporting reaction forces acting at the base of the vertical supports. Assuming the worst-case scenario, the maximum stress in the system is 0.136MPa, significantly below the 26MPa yield stress indicated in Table 1. Full calculations can be found in Appendix 2.

Functionality & Accuracy

To further validate the system, depth accuracy and field-of-view (FOV) calibration tests were performed. The depth was recorded at 5cm increments from 15cm to 45cm to establish the reliability of the coordinate system. Using a stationary subject and distance markers, the output was compared to the real distance. The distance markings were found using a tape measure, and may contain parallax or human error, but the results in Figure 7 suggest that the sensor is reliable. The test confirmed the viability of outputting real time data.

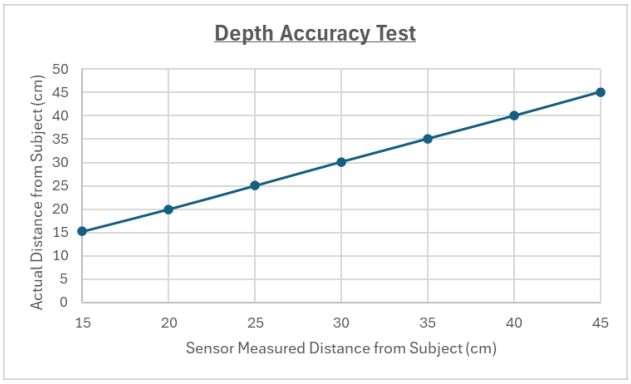
The width and height of the sensor's FOV were recorded at each interval to define the tracking window. The results of the test are shown in Figure 8, the values are used to define the distance from the camera lens to the subject that will be used for the bracket, ensuring the tracked points remain in the FOV from 0-90º flexion. A 15cm distance between the patella and lens was chosen as it captures the patellar movement across the full flexion cycle whilst allowing a buffer to account for postural sway or limb misalignment. The FOV width of 9.1cm comfortably fits the maximum mediolateral patella translation of 19.3mm observed in the population (35). The output accuracy was found to be ±1.013mm. Table 5 shows the results of using Equation 1 to calculate the required FOV from the literature and using the experimental values for 15cm. The calculated FOV angle is 50.8º which is larger than the 43.6º angle required by literature (4).

$$FOV = 2 \times \tan^{-1}\left(\frac{h}{2d}\right) \quad (1)$$

Equation 1 – h is the maximum proximal/distal translation of the patella and d is the distance from the patella to the camera lens

Table 5 – Comparison of Tested FOV Angle and Required Angle from Literature at 15 cm.

	Measured	Literature (4)
h [cm]	14.25	12
FOV Angle [º]	50.8	43.6



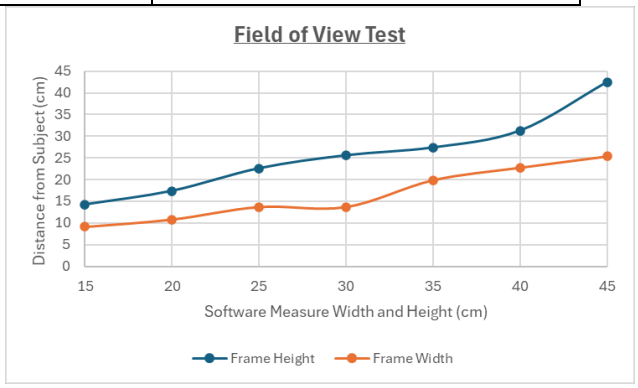


Figure 7 - Graph of Real Depth vs Measured Depth; Figure 8 – Graph indicating accuracy of FOV

Code

A separate Python code uses colour masking to isolate the specific pen marks on the patient's patella by filtering out all other background colours. As the leg moves, the encoder reads the leg flexion angle, and the Arduino sends the value to the main Python code on the laptop. The main Python code matches this data with the iPhone 3D depth data to calculate the patellar mediolateral shift throughout the knee's flexion from 0-90º. Figure 9 shows a flowchart of the code's logic; the entire code can be found in Appendix 3.

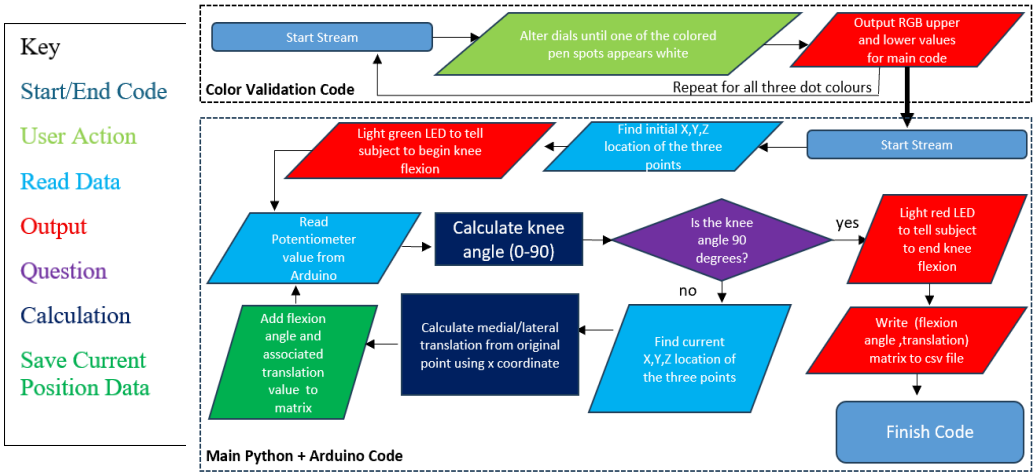


Figure 9 – Flowchart demonstrating data gathering and interpretation within the code

Conclusion

The report validated the feasibility of constructing a wearable device that quantifies patella motion to assist in maltracking diagnosis. An iPhone time-of-flight sensor was fixed 15cm from the patella; attached to a functional knee brace, with a 50.8º field of view angle, the design covered the full mean mediolateral displacement range of 19.3mm, with an accuracy of ±1.013mm. A four-point bending test, alongside a free body diagram, confirmed the safety of the supports, with a yield of ~75MPa and maximum predicted operation stress of 0.136MPa. Use of a battery was found to be unsafe, reaching temperatures of 45ºC and it was decided that a cabled connection should be used instead. An iterative design process was followed in conjunction with international standards, to produce a Class Im medical device, capable of providing an accurate, repeatable method for assessing patella tracking through dynamic motion analysis.

Appendix

A1 – Meeting Log

Date: 26.01.26
In Attendance: Khaled Gadalla, Emma Graham, Daniel Kekeh
Minutes:

- Discussed scope of the project
- Decided on weekly meeting time

Date: 27.01.26
In Attendance: John Fairhurst, Khaled Gadalla, Emma Graham, Daniel Kekeh, Joshua Lipshaw
Minutes:

- Discussed individual design ideas, alongside their limitations, advantages, accuracy and ease of manufacturability
- Voted on design to go forward with – chose time of flight sensor method

Date: 30.01.26
In Attendance: John Fairhurst, Emma Graham, Daniel Kekeh, Joshua Lipshaw
Minutes:

- Composed bill of materials for design
- Discussed potential different methods of linking Arduino to remove limitations of cable length

Date: 05.02.26
In Attendance: John Fairhurst, Khaled Gadalla, Emma Graham, Daniel Kekeh, Joshua Lipshaw, Zixuan Zhang
Minutes:

- Divided into software and hardware teams
    - Hardware – CAD design, testing, creation of fixed distance mount, analysis of mount using cantilever beam approximation to determine minimum required cross-sectional area, integration of potentiometer into brace, development of electronic housing
    - Software – Programming of reproducible procedure capable of outputting patella mediolateral shift to knee flexion angle, reading and analysis of data from Arduino
- Discussed next steps in advancing project
    - Hardware – Calculations of material thickness required
    - Software – Outline potential procedures independently to allow for comparison of ideas

Date: 12.02.26
In Attendance: John Fairhurst, Khaled Gadalla, Emma Graham, Daniel Kekeh, Joshua Lipshaw, Zixuan Zhang
Minutes:

- Discussed brace provided by Professor Stewart for use in the project
- Considered individual ideas for the further advancement of the project

Date: 20.02.26
In Attendance: John Fairhurst, Khaled Gadalla, Emma Graham, Daniel Kekeh, Zixuan Zhang
Minutes:

- Started constructing Arduino system
- Discussed dividing report writing work
- Prioritised tasks for the next week

Date: 24.02.26

In Attendance: John Fairhurst, Khaled Gadalla, Emma Graham, Daniel Kekeh, Zixuan Zhang
Minutes:

- Divided and began testing components
- Tested code on Arduino
- Started report writeup

Date: 02.03.26
In Attendance: John Fairhurst, Khaled Gadalla, Emma Graham, Daniel Kekeh, Joshua Lipshaw, Zixuan Zhang
Minutes:

- Completed section writeups
- Rehearsed presentation
- Tested code on Arduino

A2 – Free Body Diagram Calculations

Assuming a worst-case scenario where the weight of the phone acts directly through the second vertical support:

$$Moment = mg * e\ (Eccentricity) = 0.204 \times 9.81 * 35.75 = 71.5\ Nmm$$

$$Second\ Moment\ of\ Area\ (Neutral\ Axis) = I = \frac{b * t^3}{12} = \frac{15 * 15^3}{12} = 4218.75\ mm^2$$

$$\sigma_{axial} = \frac{R_b}{A} = \frac{2.0}{225mm^2} = 0.0089\ MPa \qquad \sigma_{bending} = \frac{Mc}{I} = \frac{71.5 * 7.5}{4218.75} = 0.127\ MPa$$

$$\sigma_{max} = \sigma_{axial} + \sigma_{bending} = 0.136\ MPa \ll\ \sigma_{yield}\ (Table\ 3)$$

A3 – Complete Software Code

Patella Flexion Angle:

```
const int ENC_A   = 2;
const int ENC_B   = 3;
const int LED_PIN = 5;
const int BTN_PIN = 6;

const long PPR = 128;                  // 128 PPR encoder
const long COUNTS_PER_REV = PPR;       // X1 counting (rising edge on A)

// Encoder
long rawCount = 0;
int lastA = HIGH;

// Debounce
```

```
bool lastBtnReading = HIGH;
bool btnStableState = HIGH;
unsigned long lastDebounceMs = 0;
const unsigned long DEBOUNCE_MS = 30;

// Double-press handling
const unsigned long DOUBLE_PRESS_MS = 350;
bool pendingSingle = false;
unsigned long firstPressMs = 0;

// LED pulse control (non-blocking)
bool ledPulseActive = false;
unsigned long ledPulseEndMs = 0;

// Recording output timing
const unsigned long SAMPLE_PERIOD_MS = 100; // ~10 Hz
unsigned long lastSampleMs = 0;

// State machine
enum Mode { MODE_IDLE, MODE_ARMING, MODE_RECORDING, MODE_CALIB };
Mode mode = MODE_IDLE;

unsigned long armStartMs = 0;

// Zeroing
long zeroOffset = 0;
unsigned long recordStartMs = 0;
long calibZeroOffset = 0;

void ledPulse(unsigned long durationMs) {
  digitalWrite(LED_PIN, HIGH);
  ledPulseActive = true;
  ledPulseEndMs = millis() + durationMs;
}

void ledService() {
  if (ledPulseActive && (long)(millis() - ledPulseEndMs) >= 0) {
```

```
    digitalWrite(LED_PIN, LOW);
    ledPulseActive = false;
  }
}

void updateEncoder() {
  int a = digitalRead(ENC_A);

  // X1: rising edge on A only
  if (lastA == LOW && a == HIGH) {
    bool cw = (digitalRead(ENC_B) == LOW);
    rawCount += cw ? 1 : -1;
  }
  lastA = a;
}

bool pressEvent() {
  bool reading = digitalRead(BTN_PIN);

  if (reading != lastBtnReading) {
    lastDebounceMs = millis();
    lastBtnReading = reading;
  }

  if (millis() - lastDebounceMs >= DEBOUNCE_MS) {
    if (reading != btnStableState) {
      btnStableState = reading;
      if (btnStableState == LOW) return true; // pressed
    }
  }
  return false;
}

float angleFromOffset(long offset) {
  long count = rawCount - offset;
  return ((float)count / (float)COUNTS_PER_REV) * 360.0f;
}
```

```
// Wrap angle to (-180, +180]
float wrapTo180(float deg) {
  float w = fmod(deg, 360.0f);
  if (w <= -180.0f) w += 360.0f;
  if (w > 180.0f) w -= 360.0f;
  return w;
}

bool inRange(float x, float lo, float hi) {
  return (x > lo && x < hi);
}

void startRecordingNow() {
  zeroOffset = rawCount;
  recordStartMs = millis();
  lastSampleMs = 0;

  // start indicator
  ledPulse(1000);

  mode = MODE_RECORDING;
}

void stopRecordingNow() {
  mode = MODE_IDLE;
  ledPulse(200);
}

void outputSampleIfDue() {
  unsigned long now = millis();
  if (lastSampleMs == 0 || (now - lastSampleMs) >= SAMPLE_PERIOD_MS) {
    lastSampleMs = now;

    long count = rawCount - zeroOffset;
    float angleDeg = ((float)count / (float)COUNTS_PER_REV) * 360.0f;
```

```
    // CSV: time since recording start, count, angle
    Serial.print(now - recordStartMs);
    Serial.print(',');
    Serial.print(count);
    Serial.print(',');
    Serial.println(angleDeg, 6);
  }
}

void enterCalibration() {
  // Define "0" for calibration relative to current shaft position
  calibZeroOffset = rawCount;
  mode = MODE_CALIB;
  ledPulse(300);
}

void exitCalibration() {
  mode = MODE_IDLE;
  digitalWrite(LED_PIN, LOW);
  ledPulse(300);
}

void updateCalibrationLED() {
  float ang = angleFromOffset(calibZeroOffset);
  float w = wrapTo180(ang);

  bool on =
    inRange(w, -3.0f,  3.0f) ||
    inRange(w, 42.0f, 48.0f) ||
    inRange(w, 87.0f, 93.0f);

  // In calibration mode, LED is controlled by window detection (no pulsing)
  if (!ledPulseActive) {
    digitalWrite(LED_PIN, on ? HIGH : LOW);
  }
}
```

```
void setup() {
  Serial.begin(115200);

  pinMode(ENC_A, INPUT_PULLUP);
  pinMode(ENC_B, INPUT_PULLUP);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  pinMode(BTN_PIN, INPUT_PULLUP);

  lastA = digitalRead(ENC_A);

  Serial.println("time_ms,count,angle_deg");
}

void loop() {
  ledService();
  updateEncoder();

  // Button press handling (press events only)
  if (pressEvent()) {
    unsigned long now = millis();

    if (!pendingSingle) {
      pendingSingle = true;
      firstPressMs = now;
    } else {
      // second press candidate
      if (now - firstPressMs <= DOUBLE_PRESS_MS) {
        // DOUBLE PRESS detected
        pendingSingle = false;

        if (mode == MODE_IDLE) {
          enterCalibration();
        } else if (mode == MODE_CALIB) {
          exitCalibration();
```

```
      } else if (mode == MODE_ARMING) {
        // treat double press during arming as cancel arming
        mode = MODE_IDLE;
        ledPulse(200);
      } else if (mode == MODE_RECORDING) {
        // immediate stop
        stopRecordingNow();
      }
    } else {
      // too slow, restart double-press window
      firstPressMs = now;
    }
  }
}

  // If a single press is pending and the double-press window expires, fire the
single-press action.
  if (pendingSingle && (millis() - firstPressMs > DOUBLE_PRESS_MS)) {
    pendingSingle = false;

    if (mode == MODE_IDLE) {
      // SINGLE press in IDLE -> arm recording sequence
      ledPulse(200);
      armStartMs = millis();
      mode = MODE_ARMING;
    } else if (mode == MODE_RECORDING) {
      stopRecordingNow();
    }
    // In CALIB mode, single press does nothing (by design).
  }

  // Arming: after 2 seconds, begin recording
  if (mode == MODE_ARMING) {
    if (millis() - armStartMs >= 2000) {
      startRecordingNow();
    }
  }
```

```
  // Recording output
  if (mode == MODE_RECORDING) {
    outputSampleIfDue();
  }

  // Calibration LED logic
  if (mode == MODE_CALIB) {
    updateCalibrationLED();
  }
}
```

Python Colour Masking Code – Ran before the main code to establish upper and lower bounds for the three lines drawn onto the subject's patella. The upper and lower bounds are then submitted in to the main code so the software can find the points in each frame of the live stream.

```
import numpy as np #importing necessary libraries
from record3d import Record3DStream
import cv2
from threading import Event #wait for new iphone info before next line


class ColorPickerApp:
    def __init__(self):
        self.event = Event()
        self.session = None


    def on_new_frame(self): #runs every time a new 'image' is taken by the
stream
        self.event.set()


    def nothing(self, x):
        pass


    def run(self):
        # 1. Connect to Record3D
        devs = Record3DStream.get_connected_devices() #searches points for
iPhone
        if not devs:
            print("No device found! Check your connection.")
            return
```

```
        self.session = Record3DStream()
        self.session.on_new_frame = self.on_new_frame
        self.session.connect(devs[0]) #link to phone when found

        # 2. Create the Trackbar Window
        cv2.namedWindow("Trackbars")
        cv2.createTrackbar("L-H", "Trackbars", 0, 179, self.nothing) #trackbars
create lines for user to alter
        cv2.createTrackbar("L-S", "Trackbars", 0, 255, self.nothing)
        cv2.createTrackbar("L-V", "Trackbars", 0, 255, self.nothing)
        cv2.createTrackbar("U-H", "Trackbars", 179, 179, self.nothing)
        cv2.createTrackbar("U-S", "Trackbars", 255, 255, self.nothing)
        cv2.createTrackbar("U-V", "Trackbars", 255, 255, self.nothing)

        print("Adjust sliders until only your target colored dot is WHITE.")

        while True:
            self.event.wait() #waits until next frame arrives
            rgb = self.session.get_rgb_frame() #takes the rgb image from the
iphone

            # Convert to HSV color space - distinguishes colour and brightness
            hsv = cv2.cvtColor(rgb, cv2.COLOR_RGB2HSV)

            # Get current positions of trackbars
            l_h = cv2.getTrackbarPos("L-H", "Trackbars")
            l_s = cv2.getTrackbarPos("L-S", "Trackbars")
            l_v = cv2.getTrackbarPos("L-V", "Trackbars")
            u_h = cv2.getTrackbarPos("U-H", "Trackbars")
            u_s = cv2.getTrackbarPos("U-S", "Trackbars")
            u_v = cv2.getTrackbarPos("U-V", "Trackbars")

            lower = np.array([l_h, l_s, l_v]) #converts to upper and lower
bounds of the selected colour from trackbars
            upper = np.array([u_h, u_s, u_v])

            # Create a mask that only shows colors in that range - white for
colours in the range
```

```
            mask = cv2.inRange(hsv, lower, upper)


            # Show the results
            cv2.imshow("Original RGB", cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR))
#normal image
            cv2.imshow("Mask (White = Detected)", mask) #black and white image
of selected colour


            if cv2.waitKey(1) & 0xFF == ord('q'): #waits for the quit code to
end stream
                # Print the final values to be added to main script before
closing
                print(f"\nFinal Values for this color:")
                print(f"Lower: [{l_h}, {l_s}, {l_v}]")
                print(f"Upper: [{u_h}, {u_s}, {u_v}]")
                break


            self.event.clear()


if __name__ == '__main__':
    app = ColorPickerApp()
    app.run()
```

Arduino (C++) Code – This code is uploaded onto the Arduino through Arduino IDE, it reads the data from the encoder and sends it to the main python code and takes instructions from the python code to light specific LEDs at different stages of the calibration/recording/end of the process.

```
const int ENC_A   = 2; //defining pins
const int ENC_B   = 3;
const int LED_PIN = 5; // Green LED
const int RED_LED = 4; // Red LED
const int BTN_PIN = 6; //physical button


const long PPR = 128; //128 pulses per revolution of the encoder
const long COUNTS_PER_REV = PPR;


long rawCount = 0; //holds encoder count
int lastA = HIGH;


// State : machine synced to Python
char currentState = 'M'; // M: Marking, R: Resetting, S: Started, F: Finished
```

```
void setup() {
  Serial.begin(115200); // Communication with python
  pinMode(ENC_A, INPUT_PULLUP); #sets the encoders and button as input data
  pinMode(ENC_B, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  lastA = digitalRead(ENC_A);
}
//continuous loop to run in the background
void loop() {
  // 1. Update Encoder
  int a = digitalRead(ENC_A);
  if (lastA == LOW && a == HIGH) {
    rawCount += (digitalRead(ENC_B) == LOW) ? 1 : -1;
  }
  lastA = a;


  // 2. Listen for Python Sync Commands
  if (Serial.available() > 0) {
    currentState = Serial.read(); //updates current state with the letter sent
by python
  }


  float angle = ((float)rawCount / (float)COUNTS_PER_REV) * 360.0f; //converts
encoder values to degrees


  // 3. LED Sync Logic
  if (currentState == 'R') { // RESETTING
    digitalWrite(LED_PIN, (angle > 2.0 && millis() % 500 < 250) ? HIGH : LOW);
    digitalWrite(RED_LED, LOW); //before recording has started, flashing to tell
user to straighten leg to start recording
  }
  else if (currentState == 'S') { // RECORDING
    digitalWrite(LED_PIN, HIGH); //solid green light for recording
    digitalWrite(RED_LED, LOW);
  }
  else if (currentState == 'F') { // FINISHED
```

```
    digitalWrite(LED_PIN, LOW);
    digitalWrite(RED_LED, (millis() % 400 < 200) ? HIGH : LOW); //red LED
flashes - end of test
  }

  // 4. Output Data (Synced to Python's .split(":") logic)
  static unsigned long lastTx = 0;
  if (millis() - lastTx >= 10) { // 100Hz for data send to python
    Serial.print("ANGLE:");
    Serial.println(angle); //prints what the python code will capture
    lastTx = millis();
  }
}
```

Main Python Code – The main python code takes data from the connected iPhone processes the data to show a stream on the laptop, finds the coloured points and their location and communicates with the Arduino through the C++ code to send LED signals for the user and receive encoder data. The code writes the final output of the measuring to a CSV file, being the medial/lateral translation of the patella at each knee flexion angle recorded.

```
import numpy as np #importing necessary libraries
import cv2
import serial
from record3d import Record3DStream
from threading import Event


# --- USER CONFIGURATION — setting the upper and lower limits from the colour
dial test, colours of pens can be changed for final design
SERIAL_PORT = 'COM5'  #specifies the port for the arduino communication
COLOUR_RANGES = {
    'red':   {'low': np.array([0, 120, 70]),   'high': np.array([10, 255,
255])},
    'blue':  {'low': np.array([100, 150, 0]),  'high': np.array([140, 255,
255])},
    'green': {'low': np.array([40, 100, 100]), 'high': np.array([80, 255, 255])}
//change to user values
}


class PatellarMaster:
    def __init__(self):
        self.event = Event()
        self.session = None
```

```
        self.arduino = serial.Serial(SERIAL_PORT, 115200, timeout=0.1)
        self.baselines = {}
        self.log = []

    def get_x_meter(self, px, py, depth, intrinsic):
        z = depth[py, px] #gets the depth of a given pixel at the coordinates (px,py)
        if z == 0: return None #matrix will have ‘none’ if depth can’t be detected
        fx, cx = intrinsic[0, 0], intrinsic[0, 2] #specified iphone focus length and center point
        return (px - cx) * z / fx #calculates the physical coordinates from the pixel data

    def find_dot(self, hsv, depth, intrinsic, colour):
        rng = COLOUR_RANGES[colour] #defines the range of RGB values for the specific colour point we are looking for (red,blue,green) is the colour
        mask = cv2.inRange(hsv, rng['low'], rng['high']) #hiding everything within the frame except from the colour range
        cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE) #finds the pixel value edges of the colour dot
        if cnts:
            c = max(cnts, key=cv2.contourArea) #removes any noise in the background from similar coloured objects - focuses on biggest blog(may be problematic if large coloured object in frame)
            M = cv2.moments(c) #finds the middle of the graph
            if M["m00"] > 0:
                px, py = int(M["m10"]/M["m00"]), int(M["m01"]/M["m00"]) #area of the blob returned
                return self.get_x_meter(px, py, depth, intrinsic)
        return None #if no blob returns nothing

    def run(self):
        # 1. Connect to iPhone
        devs = Record3DStream.get_connected_devices() #find the iphone port
        self.session = Record3DStream() #prepares python to receive the stream from the iphone app
        self.session.on_new_frame = lambda: self.event.set() #sets up continuous stream from iphone
        self.session.connect(devs[0]) #opens the data stream

        # Start in 'RESETTING' mode - Tells Arduino to flash Green LED
```

```
        state = "RESETTING" #waiting for patient to straighten leg
        self.arduino.write(b'R') #send the signal to the arduino to flash - will flash until angle 0 degrees
        print("STAGE 1: Move leg to 0 degrees (Wait for Solid Green LED)")

        while True:
            if not self.event.wait(0.1): continue #waits for the data from the camera

            # Read Arduino Angle
            line = self.arduino.readline().decode('utf-8').strip() #reads one line of data from the arduino
            # Check for "ANGLE:" to stay in sync with Encoder code
            angle = float(line.split(":")[1]) if "ANGLE:" in line else 0.0 #takes just the number for the angle of the potentiometer

            # Grab latest 3D Data
            depth = self.session.get_depth_frame() #takes the z depth data
            rgb = self.session.get_rgb_frame() #takes the rbg data for all coordinates
            self.event.clear() #clears the data - waits for a new frame

            intrinsic_coeffs = self.session.get_intrinsic_mat()
            intrinsic = np.array([
                [intrinsic_coeffs.fx, 0, intrinsic_coeffs.tx],
                [0, intrinsic_coeffs.fy, intrinsic_coeffs.ty],
                [0, 0, 1]
            ]) #organises coefficients into standard 3x3 matrix

            small_rgb = cv2.resize(rgb, (640, 480)) #shrinks video from 4k to a smaller processible video
            hsv = cv2.cvtColor(small_rgb, cv2.COLOR_RGB2HSV) #changes to hsv, separates colour from brightness against rgb file

            # --- STAGE 1: WAITING FOR 0° - if stage is resetting waiting for the start of data
            if state == "RESETTING":
                if angle < 2.0: # leg is essentially straight
                    # Attempt to lock baseline
```

```
                    temp_baselines = {} #temporary holds the positions of the
dot
                    for c in ['red', 'blue', 'green']: # runs the find for each
dot
                        temp_baselines[c] = self.find_dot(hsv, depth, intrinsic,
c)


                    if all(v is not None for v in temp_baselines.values()): #
ensures the test does not start unless all three coloured dots are found
                        self.baselines = temp_baselines
                        # Tell Arduino to go SOLID GREEN
                        self.arduino.write(b'S') # leg is straight - LED green
to indicate start of the recording
                        print("STAGE 2: Baseline Set. RECORDING START!")
                        state = "RECORDING"


            # --- STAGE 2: LIVE RECORDING ---
            elif state == "RECORDING":
                shifts = [] #blank matrix of zeroes to hold the translation for
each of the dots
                for c, start_x in self.baselines.items(): #goes through the
starting coordinates of the points to find the translation
                    curr_x = self.find_dot(hsv, depth, intrinsic, c) #find
current x coordinate of the dot
                    if curr_x is not None: shifts.append(curr_x - start_x) #
finds the translation of points, none ensures code doesn't crash if the dot is
lost


                if shifts: #only do the maths for the final value if we found
one of the three dots
                    avg_shift = np.mean(shifts) #takes the average shift of all
three points
                    self.log.append([angle, avg_shift * 1000]) # Store in mm at
each angle


                if angle > 88.0: # detects end of movement
                    # Tell Arduino to FLASH RED
                    self.arduino.write(b'F')
                    print("STAGE 3: 90 degrees reached. Saving data.")
                    np.savetxt("medial_lateral_shift.csv", self.log,
delimiter=",",
                               header="Angle,Shift_mm", comments='') # saves
into CSV
```

```
                    state = "FINISHED"
                    break


            cv2.imshow("Tracking Feed", cv2.cvtColor(small_rgb,
cv2.COLOR_RGB2BGR)) #live video window
            if cv2.waitKey(1) & 0xFF == ord('q'): break #q key for emergency
stop


if __name__ == "__main__":
    PatellarMaster().run()
```

References

1. Smith, B.E., Selfe, J., Thacker, D., Hendrick, P., Bateman, M., Moffatt, F., Rathleff, M.S., Smith, T.O. and Logan, P. Incidence and Prevalence of Patellofemoral Pain: A Systematic Review and Meta-Analysis. *PLoSOne.* [Online]. 2018, **13**(1), Article Number: e0190892.
2. Brenneis, M., Junker, M., Sohn, R., Braun, S., Ehnert, M., Zaucke, F., Jenei-Lanzl, Z. and Meurer, A. Patellar Malalignment Correlates with Increased Pain and Increased Synovial Stress Hormone Levels – A Cross-Sectional Study. *PLoS One.* [Online]. 2023, **18**(7), Article Number: e0289298.
3. Garza-Borjón, A., González-González, M., de la Garza-Salazar, J.F., Simental-Mendía, M. and Acosta-Olivo, C. Understanding the Patho-Anatomy of Patellofemoral Pain: A CrucialFoundation for Comprehensive Management. *Orthopaedic Reviews (Pavia).* [Online]. 2024, **16**, Article Number: 125840.
4. Jibri, Z., Jamieson, P., Rakhra, K.S., Sampaio, M.L. and Dervin, G. Patellar Maltracking: An Update on the Diagnosis and Treatment Strategies. *Insights Imaging*. [Online]. 2019, **10**(1), pp. 65-76.
5. Katchburian, M.V., Bull, A.M.J., Shih Y-F., Heatley, F.W. and Amis, A.A. Measurement of Patellar Tracking: Assessment and Analysis of the Literature. *Clinical Orthopaedics and Related Research*. [Online]. 2003, **412**, pp. 241-259.
6. NHS UK. *Understanding UK MDR 2002 Regulations for Medical Devices*. [Online]. 2025. Available from: https://www.digitalregulations.innovation.nhs.uk/regulations-and-guidance-for-developers/all-developers-guidance/understanding-uk-mdr-2002-regulations-for-medical-devices/
7. Flood, D., McCaffery, F., Casey, V., McKeever, R. and Rust, P. A Roadmap to ISO 14971 Implementation. *Journal of Software: Evolution and Process*. [Online]. 2015, **27**(5), pp. 319–36.
8. Simsekler, M.C.E., Kaya, G.K., Ward, J.R. and Clarkson, P.J. Evaluating Inputs of Failure Modes and Effects Analysis in Identifying Patient Safety Risks. *International Journal of Health Care Quality Assurance.* [Online]. 2019, **32**(1), pp. 191–207
9. Chhabra, A., Subhawong, T.K. and Carrino, J.A. A Systematised MRI Approach to Evaluating the Patellofemoral Joint. *Skeletal Radiology.* [Online]. 2010, **40**(4), pp. 375-387.
10. International Standards Office. ISO 20685-1:2018. *3D Scanning Methodologies for Internationally Compatible Anthropometric Databases.* Geneva: ISO, 2018.
11. Tondo, D.R., Riley, C. and Morgenthal, G. Characterisation of the iPhone LiDAR-Based Sensing System for Vibration Measurement and Modal Analysis. *Sensors (Basel).* [Online]. 2023, **23**(180), Article Number: 7832.
12. Mouser. *ENA1J-B28-L00128L Datasheet*. [Online]. 2026. [Accessed 24 February 2026]. Available from: https://www.mouser.co.uk/datasheet/3/40/1/ENC1J.pdf
13. International Standards Office. ISO 9241-210:2019. *Ergonomics of Human-System Interaction.* Geneva: ISO, 2019.
14. British Standards Institution. BS EN ISO 17664-1:2021. *Processing of Healthcare Products.* London: BSI, 2021.
15. Farnell. *Technical Datasheet PLA*. [Online]. 2025. [Accessed 24 February 2026]. Available from: https://www.farnell.com/datasheets/2310522.pdf
16. Grood, E.S. and Suntay, W.J. A Joint Coordinate System for the Clinical Description of Three-Dimensional Motions: Application to the Knee. *Journal of Biomechanical Engineering.* [Online]. 1983, **105**(2), pp. 136-144.

17. British Standards Institution. BS EN ISO 7250:2017. *Basic Human Body Measurements for Technological Design*. London: BSI, 2017.
18. British Standards Institution. BS EN ISO 10993:2025. *Biological Evaluation of Medical Devices*. London: BSI, 2025.
19. British Standards Institution. BS EN ISO 10328:2016. *Structural Testing of Lower Limb Prostheses.* London: BSI, 2016
20. International Standards Office. ISO 6385:2016. *Ergonomics Principles in the Design of Work Systems*. Geneva: ISO, 2016.
21. International Standards Office. ISO 13485:2016. *Medical Devices – Quality Management Systems.* Geneva: ISO, 2016.
22. International Electrotechnical Commission. PD IEC/TR 62366:2016. *Medical Devices – Guidance on the Application of Usability Engineering to Medical Devices.* Geneva: IEC, 2016.
23. International Standards Office. ISO 15223-1:2021. *Medical Devices – Symbols to be Supplied by the Manufacturer.* Geneva: ISO, 2021.
24. International Standards Office. ISO 62304:2006. *Medical Device Software*. Geneva: ISO, 2006.
25. International Electrotechnical Commission. IEC 60601-1:2015. *Medical Electrical Equipment.* Geneva: IEC, 2015.
26. International Standards Office. ISO 14971:2019. *Medical Devices – Application of Risk Management to Medical Devices.* Geneva: ISO, 2019.
27. International Electrotechnical Commission. IEC 62133:2017. *Safety Testing for Lithium Ion Batteries.* Geneva: IEC, 2017.
28. British Standards Institution. BS EN ISO 15883-1:2025. *Washer-Disinfectors.* London: BSI, 2025.
29. Stratasys. Data Sheet. Eden Prairie. Report: ASTM D638. 2018.
30. MatWeb. Data Sheet. Material Property Data. 2019.
31. Bambu Labs. PLA Basic Filament. [Online]. 2026. [Accessed 27 February 2026]. Available from: https://uk.store.bambulab.com/products/pla-basic-filament
32. International Standards Office. ISO 14937:2009. *Sterilisation of Healthcare Products*. Geneva: ISO, 2009.
33. Vaňková, E., Kašparová, P., Khun, J., Machková, A., Julák, J., Sláma, M., Hodek, J., Ulrychová, L., Weber, J., Obrová, K., Kosulin, K., Lion, T. and Scholtz, V. Polyactic Acid as a Suitable Material for 3D Printing of Protective Masks in times of COVID-19 Pandemic. *PeerJ.* [Online]. 2020, **8**, Article Number: e10259.
34. Medical Devices Regulations 2002. (Annex IX). [Online] London: The Stationery Office. Available from: https://www.legislation.gov.uk/eudr/1993/42/annex/IX/2020-12-31
35. Frings, J., Dust, T., Krause, M., Ohlmeier, M., Frosch, K-H., Adam, G., Warncke, M., Maas, K-J. and Henes, F.O. Objective Assessment of Patellar Maltracking with 3T Dynamic Magnetic Resonance Imaging: Feasibility of a Robust and Reliable Measuring Technique. *Scientific Reports.* [Online]. 2020, **10**, Article Number: 16770